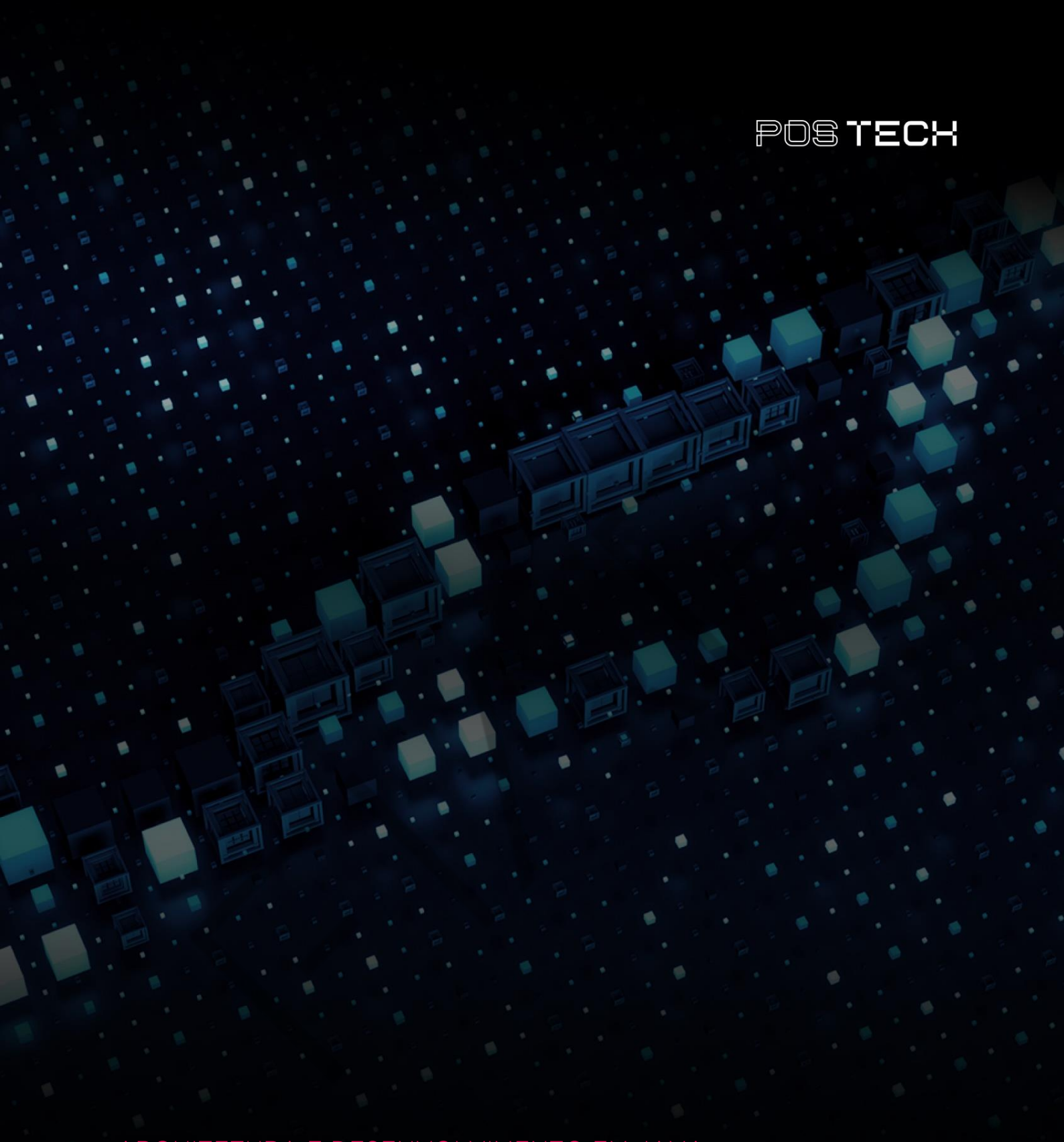


PÓS TECH

ARQUITETURA E DESENVOLVIMENTO EM JAVA

FASE 3 | AULA 03 -

PRODUZINDO E CONSUMINDO MENSAGENS COM KAFKA

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3

HANDS ON.....4

SAIBA MAIS 5

O QUE VOCÊ VIU NESTA AULA?.....10

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VEM POR AÍ?

Nesta aula, colocaremos em prática a configuração e utilização do Apache Kafka com Java para produção e consumo de mensagens. Começaremos com o setup do ambiente Kafka, que pode ser feito localmente ou via Docker.

Avançaremos para a configuração de um projeto Java, adicionando as dependências essenciais e organizando o código em pacotes dedicados para producers e consumers. Exploraremos as propriedades fundamentais de configuração para ambos, como serialização de mensagens e conexão com os brokers do Kafka.

Também implementaremos um Producer em Java, responsável por enviar mensagens a um tópico, e um Consumer, que será configurado para ler essas mensagens, utilizando a API oficial do Kafka. Por fim, veremos como testar a interação entre producer e consumer, validando o fluxo de mensagens e analisando cenários com múltiplos consumidores.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

HANDS ON

Vamos aprender os passos necessários para configurar e utilizar o Apache Kafka em conjunto com Java. Discutiremos os detalhes para o setup do ambiente, tanto localmente quanto com o uso de Docker, além da criação de tópicos com parâmetros específicos, como partições e fator de replicação.

Em seguida, abordaremos como configurar um projeto Java para integração com o Kafka desde a adição das dependências necessárias até a estruturação do código em pacotes organizados. Explicaremos os detalhes de configuração para um Producer, incluindo as propriedades.

Além disso, discutiremos o fluxo básico de produção e consumo de mensagens, descrevendo como um Producer envia mensagens para um tópico e como um Consumer lê essas mensagens em tempo real.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SAIBA MAIS

Partições dos tópicos no Apache Kafka

Partições são divisões físicas dos tópicos no Apache Kafka, projetadas para aumentar o paralelismo e a escalabilidade do sistema. Cada tópico pode ser dividido em uma ou mais partições, permitindo que os eventos sejam distribuídos e armazenados de forma segmentada.

Essa divisão possibilita que várias mensagens de um mesmo tópico sejam processadas simultaneamente, já que diferentes partições podem ser manipuladas por consumidores distintos em paralelo. Isso é especialmente útil em cenários de alta demanda, onde grandes volumes de dados precisam ser gerenciados com eficiência.

As partições também determinam a ordem das mensagens. Em uma partição, a ordem é garantida, o que é essencial para cenários que dependem de processamento sequencial. No entanto, entre diferentes partições de um mesmo tópico, essa ordem pode variar.

Além de promover o paralelismo, as partições são fundamentais para o balanceamento de carga. O Kafka distribui essas partições entre os brokers no cluster, permitindo que os dados sejam armazenados e acessados de forma distribuída, aumentando a resiliência e a capacidade de armazenamento do sistema.

Com as partições, o Kafka consegue atender a arquiteturas de alto desempenho, processando grandes volumes de mensagens de maneira escalável e eficiente, sem comprometer a integridade dos dados.

Para entender também sobre partições, precisamos falar sobre o offset, que é a posição única que identifica cada mensagem em uma partição no Apache Kafka. Ele funciona como um marcador que indica onde uma mensagem está localizada na sequência de eventos armazenados, permitindo que os consumidores acompanhem o progresso da leitura em uma partição.

Cada mensagem em uma partição recebe um offset atribuído sequencialmente no momento em que é publicada. Isso garante que a ordem das mensagens seja preservada dentro da partição. Quando um consumidor lê mensagens, ele registra o

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

offset da última mensagem processada, o que permite retomar a leitura do ponto exato em caso de interrupções ou falhas.

O gerenciamento de offsets é essencial para a flexibilidade do consumo de mensagens. Os consumidores podem optar por iniciar a leitura do começo de uma partição, continuar de onde pararam ou até mesmo saltar para mensagens mais recentes, dependendo das necessidades da aplicação.

Esse mecanismo garante não apenas a confiabilidade na entrega das mensagens, mas também permite cenários como reprocessamento de dados ou recuperação em situações de erro, tornando o Kafka confiável para sistemas baseados em eventos.

Quando estamos estudando sobre o Kafka, é comum utilizarmos o termo“brokers”. Mas o que são esses brokers? São os servidores que compõem o cluster do Apache Kafka, responsáveis por armazenar e gerenciar os dados publicados nos tópicos. Cada broker no cluster desempenha um papel fundamental na distribuição e no gerenciamento eficiente de mensagens, garantindo que os dados sejam acessíveis e processados de forma confiável.

Um broker armazena uma ou mais partições de diferentes tópicos e o Kafka distribui essas partições entre os brokers para garantir balanceamento de carga e escalabilidade. Além disso, um broker pode atuar como líder para certas partições, coordenando as operações de leitura e escrita, enquanto outros brokers mantêm réplicas dessas partições para garantir alta disponibilidade e resiliência.

Os produtores enviam mensagens diretamente para os brokers que armazenam as partições relevantes e os consumidores leem essas mensagens também dos brokers, utilizando o offset para localizar o ponto exato na partição. Em um cluster com múltiplos brokers, o Kafka é capaz de escalar horizontalmente para lidar com grandes volumes de dados, distribuindo a carga de trabalho entre os servidores.

A coordenação entre os brokers é fundamental para manter a integridade e a consistência do sistema, seja por meio do Zookeeper ou, no futuro, com a implementação completa do KRaft. Os brokers são, portanto, o núcleo do Apache Kafka, garantindo que o sistema funcione de forma escalável, resiliente e eficiente, mesmo em cenários de alta complexidade.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Já o Zookeeper é um componente utilizado pelo Apache Kafka para a coordenação e gerenciamento dos brokers dentro de um cluster. Ele atua como um serviço centralizado que mantém informações sobre o estado do cluster, garantindo que todos os brokers operem de maneira sincronizada e consistente.

Algumas de suas principais responsabilidades são gerenciar a lista de brokers ativos, ajudar na eleição de líderes para as partições e armazenar metadados críticos, como configurações dos tópicos e informações sobre os consumidores. Essa coordenação é essencial para garantir que o Kafka funcione de maneira confiável, mesmo em ambientes distribuídos com múltiplos brokers.

No entanto, o Kafka está em processo de transição para um novo modelo de coordenação nativo chamado KRaft (Kafka Raft), que elimina a dependência do Zookeeper. Ele foi introduzido para simplificar a arquitetura do Kafka, tornando o sistema mais eficiente, integrando a coordenação e o gerenciamento diretamente no próprio sistema e eliminando a necessidade de um componente externo.

O KRaft utiliza o algoritmo de consenso Raft para coordenar os brokers no cluster, garantindo que os metadados, como informações sobre tópicos, partições e estados de líderes, sejam mantidos de forma consistente e distribuída entre os brokers. Essa integração melhora o desempenho, reduz a complexidade operacional e facilita o gerenciamento do Kafka, especialmente em clusters grandes e dinâmicos.

Além disso, o KRaft oferece maior resiliência ao reduzir os pontos de falha potenciais, já que todas as operações críticas agora são gerenciadas dentro do Kafka. Isso simplifica a implantação e o monitoramento do sistema, tornando-o mais eficiente para atender às demandas modernas de alta escalabilidade e confiabilidade.

Enquanto o Zookeeper desempenhou um papel essencial na evolução do Kafka, o KRaftdá um passo significativo em direção a uma arquitetura mais autossuficiente e preparada para o futuro, alinhada às demandas crescentes de escalabilidade e simplicidade operacional, mantendo sua posição como uma das plataformas mais robustas e eficientes para streaming de eventos e pipelines de dados distribuídos.

Vamos entender o que é o algoritmo de consenso Raft utilizado pelo KRaft, um algoritmo de consenso amplamente utilizado para coordenar sistemas distribuídos, garantindo que múltiplos servidores em um cluster concordem sobre o estado

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

compartilhado, mesmo em cenários de falhas. Ele é projetado para ser simples, compreensível e altamente confiável, sendo uma escolha popular para gerenciar a consistência de dados em sistemas distribuídos.

O algoritmo opera com base em três papéis principais para os servidores no cluster: líder, candidato e seguidor. O líder é responsável por coordenar as operações e replicar as informações para os seguidores. Se o líder falhar, o algoritmo realiza uma nova eleição para escolher um substituto de forma eficiente, garantindo a continuidade das operações.

A principal função do Raft é manter um log replicado, onde todas as mudanças no estado do sistema (como atualizações de dados ou metadados) são registradas e propagadas para todos os servidores. Isso assegura que todos os membros do cluster tenham uma visão consistente e atualizada do estado, mesmo que alguns servidores apresentem falhas temporárias.

Ele resolve o desafio crítico de coordenação em sistemas distribuídos, garantindo que decisões importantes sejam tomadas de forma consistente e que os dados sejam replicados de maneira confiável entre os servidores do cluster.

Com isso, vemos que o Apache Kafka é projetado para atender às demandas de sistemas modernos, oferecendo uma combinação poderosa de alta performance, persistência, resiliência e escalabilidade.

A alta performance do Kafka é alcançada por meio de um design otimizado para throughput elevado e baixa latência. Ele é capaz de processar milhões de mensagens por segundo, garantindo um fluxo contínuo de dados mesmo em ambientes de alta escala. Isso o torna ideal para aplicações que demandam respostas rápidas, como análises em tempo real e integração de grandes volumes de eventos.

No quesito persistência, o Kafka se destaca pela capacidade de armazenar mensagens em disco por um período configurável. Essa funcionalidade não apenas garante a integridade dos dados em caso de falhas, mas também permite que consumidores processem mensagens no ritmo que lhes convêm, seja em tempo real ou retrospectivamente.

A resiliência é outra característica fundamental do Kafka, possibilitada pela replicação automática de dados entre brokers. Cada partição de um tópico pode ter réplicas distribuídas em diferentes brokers, garantindo que os dados permaneçam

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

acessíveis mesmo em caso de falhas de hardware ou interrupções em um broker específico.

Por fim, a escalabilidade do Kafka permite que novos brokers sejam adicionados ao cluster de forma transparente, sem a necessidade de downtime. Isso facilita a expansão do sistema para lidar com crescimentos na carga de trabalho ou no volume de dados, tornando o Kafka uma solução confiável e eficiente para arquiteturas que exigem flexibilidade e desempenho contínuo.

Com essas características combinadas, o Kafka se consolida como uma ferramenta robusta para lidar com os desafios de processamento e gerenciamento de dados em sistemas distribuídos.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, aprendemos a configurar e utilizar o Apache Kafka com Java para criar soluções de mensageria em sistemas distribuídos. Exploramos o processo de configuração do ambiente, além da criação de tópicos com definições específicas de partições e fator de replicação.

Também discutimos a estruturação de um projeto Java para integrar com o Kafka, abordando a configuração das propriedades essenciais para Producers e Consumers.Vimos o funcionamento de um Producer, responsável por publicar mensagens em tópicos, e de um Consumer, encarregado de consumir essas mensagens, com explicações detalhadas sobre a lógica de processamento.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

REFERÊNCIAS

APACHE KAFKA. Apache Kafka Documentation. 2024. Disponível em: https://kafka.apache.org/. Acesso em: 21 jan. 2025.

AWS. Benefícios do Message Queue. 2024. Disponível em: https://aws.amazon.com/pt/message-queue/benefits/. Acesso em: 21 jan. 2025.

RED HAT. O que é Apache Kafka? 2024. Disponível em: https://www.redhat.com/pt-br/topics/integration/what-is-apache-kafka. Acesso em: 21 jan. 2025.

FIAP

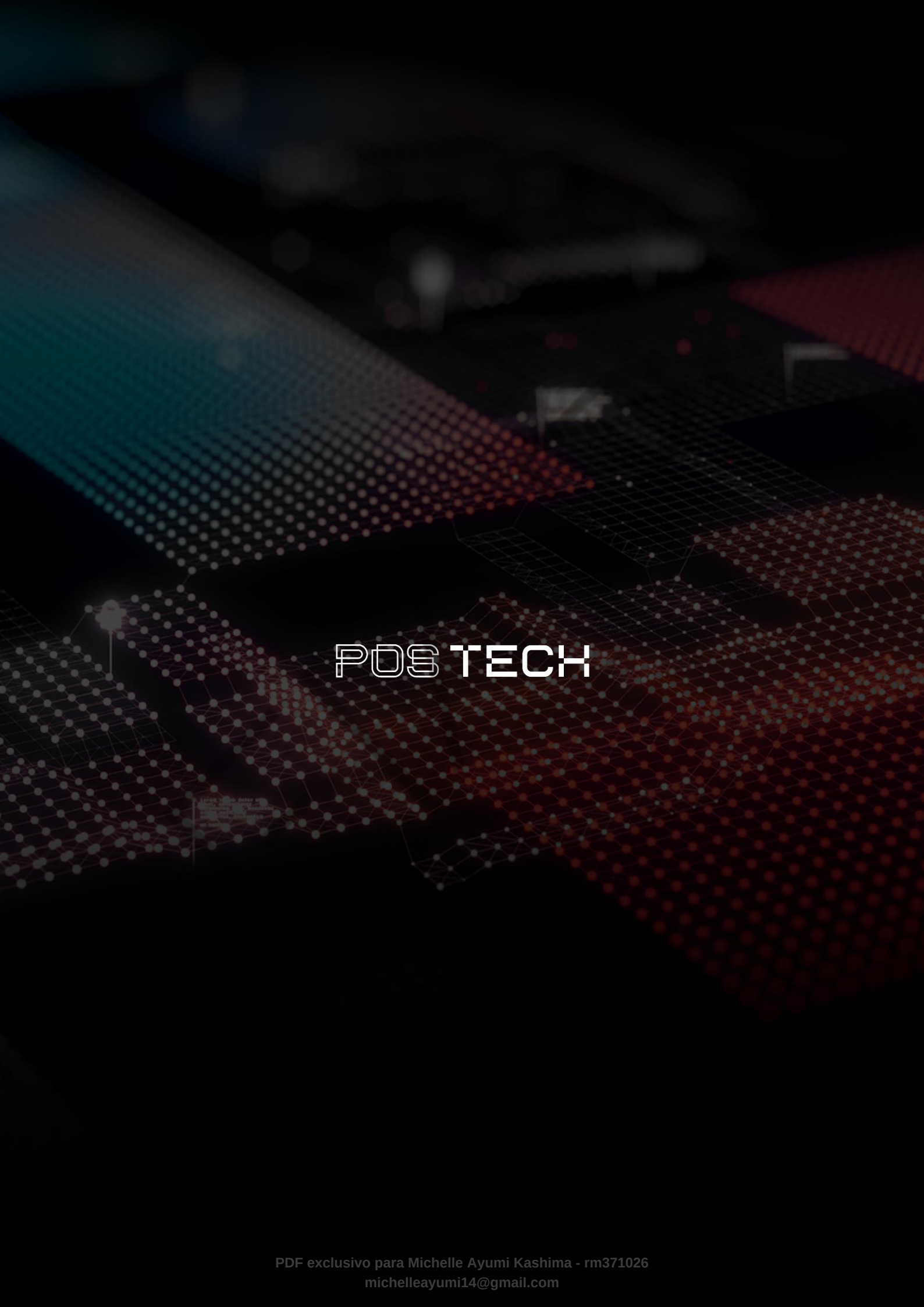
PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

PALAVRAS-CHAVE

Mensageria. Message Broker. Fila de Mensagens. Pub/Sub. Producers. Consumers. Queues. Exchanges. Roteamento de Mensagens. AMQP. RabbitMQ. Apache Kafka. AWS SQS. Escalabilidade. Baixo Acoplamento. Persistência de Mensagens. Garantia de Entrega. Resiliência. Comunicação Assíncrona. Sistemas Distribuídos. Eventos. JSON. XML. Confiabilidade. Cloud-Native. Processamento em Lote. Boas Práticas. Integração de Sistemas. Protocolos de Mensageria. Paginação de Mensagens. Alto Desempenho.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com



PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com